



Norma de Estructura de Repositorios, Control de Versiones y Evaluación

Sistemas Distribuidos 2026-B — Programa de Ingeniería de Sistemas

Documento normativo. Define la estructura obligatoria de los repositorios del proyecto, el régimen de ramas, las reglas de trazabilidad y los criterios con los que se evalúa cada corte.

Lo aquí establecido no es orientativo. Cada regla lleva un identificador y una forma de verificación objetiva; la evaluación se realiza ejecutando esas verificaciones sobre los repositorios del equipo, no sobre la impresión general del avance.

1. Objeto y alcance

1.1. Esta norma aplica a todos los repositorios del proyecto de cada equipo, en las tres categorías definidas en el numeral 3.

1.2. Prevalece sobre cualquier convención interna del equipo. Un equipo puede añadir reglas más estrictas; no puede relajar las de este documento.

1.3. El desconocimiento de esta norma no es causal de exoneración. Se publica antes del inicio del periodo de construcción.

1.4. Toda verificación se hace sobre lo que está publicado en el repositorio remoto. El trabajo que exista únicamente en la máquina de un integrante no es evidencia y no se evalúa.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



2. Definiciones

Los términos siguientes tienen, dentro de este documento, el significado que aquí se les da.

Término	Definición
Rama permanente	Rama que representa un ambiente y nunca se elimina: main, qa, develop
Rama hija	Rama de vida corta creada a partir de una rama permanente, que retorna a ella por Pull Request y luego se elimina
Promoción	Acto de hacer llegar un cambio ya validado de un ambiente al siguiente
Re-aplicación	Forma en que se promueve: el commit se vuelve a aplicar sobre una rama hija del destino, obteniendo un identificador nuevo
Rastro de promoción	La línea (cherry picked from commit <sha>) que deja git cherry-pick -x
Dominio	Contexto delimitado del negocio, con lenguaje, reglas y datos propios
Pieza	Cada uno de los repositorios en que se materializa un dominio: db, api, portal, app
Transversal	Repositorio que resuelve una preocupación que atraviesa todos los dominios
Canal	Medio por el que el usuario consume el sistema: web o móvil
Evidencia	Artefacto publicado y verificable que demuestra el cumplimiento de una regla



3. Categorías de repositorio

3.1. Todo repositorio del equipo pertenece a una y solo una de estas categorías.

Categoría	Identificación	Régimen de ramas
A. Prototipo	Repositorio inicial del equipo	Sin restricciones
B. Documentación	<abbr>-docs	Una rama permanente: main
C. Código	<abbr>-<resto>	Tres ramas permanentes

3.2. El repositorio de categoría A corresponde al prototipo del primer corte. Terminado ese corte deja de recibir trabajo y se conserva como registro histórico. No se evalúa después del primer corte.

3.3. El repositorio de categoría B conserva la gobernanza, el modelo de dominio, la arquitectura, los requisitos y las historias de usuario. No contiene código ejecutable del producto.

3.4. Los repositorios de categoría C contienen el sistema. Su estructura y nomenclatura están fijadas en el numeral 4.

4. Topología

4.1. La cantidad **mínima obligatoria** de repositorios de categoría C se determina por la fórmula:

$$\text{Total} = 5 + N \times (2 + C)$$

N = cantidad de dominios del proyecto

C = 1 si el sistema tiene un solo canal

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



2 si tiene canal web y canal movil

4.2. Todos los repositorios que arroja la fórmula **deben existir**: los cinco transversales del numeral 4.3 y, por cada dominio, las piezas del numeral 4.4 según sus canales. No se admite un número menor ni la fusión de dos piezas en un mismo repositorio.

4.2.1. Lo **opcional** es agregar. Un equipo puede crear repositorios **adicionales** a los obligatorios — una librería de contratos compartidos, un repositorio de pruebas de extremo a extremo, un segundo *worker*— siempre que:

Condición	Por qué
Lo registre en un ADR antes de crearlo, con la necesidad que resuelve	una pieza sin justificación es complejidad sin dueño
Respete la nomenclatura del numeral 4.5	los repositorios se auditan por nombre
No absorba la responsabilidad de una pieza obligatoria	un adicional agrega, no reemplaza
Quede incluido en <i>-infra</i> si es ejecutable	lo que no se compone no existe en el sistema

4.2.2. Cada equipo decide además, y registra en un ADR:

Decisión	Alcance	Opciones
Lenguaje	por servicio	Go, Java, Python o C#
Motor de base de datos	por dominio	PostgreSQL o MongoDB
Herramienta de migraciones	por dominio	Liquibase o Flyway con PostgreSQL; Liquibase con MongoDB
<i>Framework</i> de interfaz	por proyecto	React o Angular

Un mismo sistema puede combinar lenguajes y motores. Lo que no se combina es la forma: todas las piezas de un mismo tipo siguen la misma arquitectura, cualquiera sea su lenguaje. La estructura esperada de cada tipo y lenguaje está en los anexos (numeral 19).

4.2.3. Todo ADR exigido por esta norma registra, como mínimo:



Elemento	Contenido
Contexto	El problema y las restricciones que obligan a decidir
Opciones	Al menos dos alternativas reales, no una opción y su caricatura
Criterio dominante	El factor que decidió: conocimiento del equipo, consistencia requerida, costo de operación, latencia...
Costo aceptado	Lo que la opción elegida sacrifica, dicho sin eufemismos
Consecuencias	Qué cambia en el sistema y qué queda por vigilar

Un ADR sin criterio dominante ni costo aceptado es una declaración, no una decisión, y no satisface la exigencia.

4.3. Repositorios transversales

Son exactamente cinco, uno por proyecto, independientemente de la cantidad de dominios. Los cinco son obligatorios.

Repositorio	Responsabilidad única
<abbr>-api-gateway	Puerta única: enrutamiento, filtro de credencial, límites de tasa, CORS
<abbr>-worker	Procesamiento asíncrono y trabajos programados
<abbr>-workflow	Orquestación de procesos que atraviesan dominios
<abbr>-infra	Ensamblaje, ambientes, secretos y observabilidad
<abbr>-front	Contenedor que empaqueta las interfaces de dominio

El gateway es la única entrada al sistema. Si un cliente llama directamente a un servicio de dominio, el gateway dejó de ser una puerta y pasó a ser un adorno: la autenticación, el control de tasa y la observabilidad quedan sin efecto para esa llamada.



El **worker** contiene todo lo que ocurre sin que nadie lo pida: avisos de vencimiento, cálculo de multas, reindexaciones, correos, reportes programados. Si el sistema hace algo "cada noche" o "cuando pasen tres días", vive aquí. No expone interfaz de negocio.

El **workflow** implementa los procesos que tocan varios dominios, y sobre todo **qué se deshace cuando un paso falla**. Esa es su razón de ser: sin compensaciones, un proceso distribuido que falla a la mitad deja el sistema en un estado inconsistente que nadie sabe reparar.

La **infraestructura** ensambla el sistema. Con la llegada de los repositorios -db, deja de contener las bases y pasa a componerlas: es la diferencia entre ser el dueño de las piezas y ser quien las conecta.

El **contenedor de interfaz** presenta como una sola aplicación lo que son varias piezas independientes. Concentra lo que no debe repetirse: el cliente HTTP, la sesión, la navegación y los componentes comunes.

4.4. Repositorios de dominio

Por **cada** dominio del proyecto existen obligatoriamente:

Repositorio	Responsabilidad única
<abbr>-<dominio>-db	La base de datos del dominio, completa
<abbr>-<dominio>-api	El servicio que implementa el dominio
<abbr>-<dominio>-portal	Interfaz web del dominio. Solo si hay canal web
<abbr>-<dominio>-app	Interfaz móvil del dominio. Solo si hay canal móvil

4.5. Nomenclatura

4.5.1. Todo repositorio de categoría C se nombra según una de estas dos formas, sin excepción:



<abbr>-<transversal>
<abbr>-<dominio>-<pieza>

4.5.2. <abbr> es la abreviatura asignada al proyecto. <transversal> es uno de los cinco valores del numeral 4.3. <pieza> es uno de: db, api, portal, app.

4.5.3. Los tres segmentos se escriben en minúscula, en kebab-case, sin guiones bajos, sin acentos y sin mayúsculas.

4.5.4. Quedan **expresamente prohibidas** las siguientes formas, por corresponder a convenciones distintas a la norma:

Forma prohibida	Razón
ms-<dominio>, svc-<dominio>	El prefijo del repositorio es el abbr, no el tipo de componente
Un repositorio único de migraciones	Cada base es dueña de su propia estructura (regla 7.2)
<dominio>_schema dentro de una base compartida	Esquemas separados en una sola base no son base por dominio (regla 7.1)
Portales nombrados por rol o perfil	La interfaz se organiza por dominio, no por tipo de usuario
<abbr>-<dominio>-front	La interfaz de dominio se nombra por canal: portal o app

4.5.5. El nombre del repositorio se fija al crearlo y no se cambia. Un cambio de nombre lo retira automáticamente de las reglas de protección de la organización, que apuntan por nombre exacto.

5. Contenido mínimo obligatorio por repositorio



Un repositorio que no contenga lo aquí listado se considera incompleto, con independencia de la calidad de lo que sí contenga.

5.1. Todo repositorio de categoría C

Artefacto	Exigencia
README.md	Propósito de la pieza, cómo levantarla, de qué depende
.github/CODEOWNERS	Presente y sin errores de sintaxis
.gitignore	Adecuado al lenguaje utilizado
.env.example	Todas las variables requeridas, sin valores reales
.github/workflows/ci.yml	Compila y ejecuta las pruebas en cada Pull Request (Anexo I)
Ramas main, qa, develop	Las tres existen

5.2. Repositorio -db

Estructura y reglas: **Anexo A** (PostgreSQL) y **Anexo B** (MongoDB).

El repositorio se organiza por **familia de sentencia**, en el orden en que se ejecutan:

Carpeta	Familia	Contenido
01_ddl/	DDL — definición	extensiones, esquema, tipos, tablas, llaves, vistas, funciones, procedimientos, disparadores, índices
02_dml/	DML — manipulación	datos semilla, actualizaciones, borrados, <i>upserts</i> , parches de datos
03_dcl/	DCL — control de acceso	roles, permisos, políticas por fila
04_tcl/	TCL — control de transacciones	bloques transaccionales, recuperaciones manuales, etiquetas de versión



05_rollback/	reversión	espejo de cada migración que revierte con un archivo (en Flyway, U<versión>__<descripción>.sql)
changelog/ o flyway.toml	entrada	Liquibase: changelog-master.yaml, único punto de entrada. Flyway: la configuración con las carpetas de cada familia
deploy/	instancia	la instancia propia del dominio y su ejecutor de migraciones

Artefacto	Exigencia
Definición del motor	Imagen, versión fija, configuración, volumen, chequeo de salud
Esquema	DDL completo del dominio
Migraciones	Versionadas con Liquibase o Flyway (numeral 4.2.2), ordenadas, idempotentes, cada una con su reversión
Datos semilla	Mínimos para levantar el dominio en desarrollo
Roles	Sin contraseña; los usuarios con credenciales los crea la infraestructura
Diccionario de datos	Tablas, columnas, tipos, restricciones y su significado

5.2.1. El esquema de un dominio vive aquí y **únicamente** aquí. Su presencia en el repositorio -api constituye falta grave (numeral 13).

5.2.2. Las tablas se crean **sin llaves foráneas** en 03_tables; las llaves se agregan en 04_alter.

5.2.3. Una migración ya aplicada —un *changeset* de Liquibase o un archivo V de Flyway— **nunca se edita**. Una corrección es una migración nueva. Las dos herramientas lo detectan: el *checksum* deja de coincidir.



5.2.4. En MongoDB varios conceptos cambian —no hay llaves foráneas, la estructura es un validador, la unicidad la dan los índices y cada *changeset* debe ser idempotente por sí mismo—. El Anexo B detalla las diferencias. Copiar la estructura relacional sobre Mongo **no** cumple esta norma.

5.2.5. Convenciones del esquema relacional (detalle en el Anexo A):

Regla	Por qué
Tablas en singular y <i>snake_case</i> ; restricciones e índices con nombre explícito (<i>fk_</i> , <i>chk_</i> , <i>idx_</i>)	Un nombre generado por el motor no se puede referenciar en una migración posterior
Toda llave foránea declara ON DELETE y tiene su índice	Sin índice, borrar o actualizar el padre recorre la tabla hija completa
Estados con CHECK , no con ENUM del motor	Agregar un valor a un ENUM es un cambio de esquema difícil de revertir
Texto como text con CHECK de longitud; dinero como bigint en unidades menores	El límite es una regla de negocio visible; el dinero no se redondea
Datos semilla idempotentes (<i>upsert</i>)	Reaplicar la semilla no puede duplicar ni fallar

5.2.6. Un cambio que rompería al servicio desplegado —renombrar o eliminar una columna, cambiar su tipo— se hace en **dos releases** (*expandir y contraer*): primero se agrega lo nuevo y el servicio escribe en ambos; cuando ningún servicio desplegado usa lo viejo, se retira.

5.2.7. Un índice sobre una tabla con datos se crea **CONCURRENTLY**, en una migración propia: en Liquibase, con **runInTransaction: false**; en Flyway, con el bloqueo transaccional desactivado en **flyway.toml** (Anexo A). Crearlo dentro de una transacción bloquea las escrituras de la tabla mientras dura.

5.2.8. El repositorio incluye su verificación de reconstrucción (**db-ci.yml**, Anexo A): el esquema completo se construye desde una base **vacía**, se revierte entero —con **rollback** en Liquibase, con los scripts **U** en Flyway— y se vuelve a aplicar. Un esquema que solo funciona donde fue parchado a mano impide crear el siguiente ambiente.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



5.3. Repositorio -api

Artefacto	Exigencia
Modelo de dominio	Entidades, objetos de valor, invariantes y puertos
Capa de aplicación	Casos de uso explícitos, uno por operación de negocio
Adaptadores	HTTP, persistencia y mensajería, separados del dominio
Contrato publicado	Especificación OpenAPI o equivalente, versionada
Pruebas	Del dominio y de los casos de uso; del adaptador HTTP; de integración del adaptador de persistencia contra el motor real
Dockerfile	Construcción reproducible

5.3.1. El modelo de dominio no importa bibliotecas de framework web, de acceso a datos ni de mensajería. La dependencia va siempre de afuera hacia adentro.

5.3.2. El servicio sigue **arquitectura hexagonal**, con esta forma en cualquier lenguaje (estructura por lenguaje en el **Anexo C**):

Capa	Contenido	Puede depender de
Dominio	entidades, objetos de valor, invariantes	nada del proyecto
Puertos de entrada	lo que el servicio ofrece	dominio
Puertos de salida	lo que el servicio necesita	dominio
Casos de uso	una implementación por operación de negocio	puertos y dominio
Adaptadores de entrada	HTTP → caso de uso; error de dominio → código de estado	puertos de entrada
Adaptadores de salida	base de datos, colas, otros servicios	puertos de salida
Raíz de composición	el único lugar que conoce todos los tipos concretos	todo



5.3.3. En Java y C#, el núcleo va en un módulo o proyecto **separado que no declara el framework como dependencia**, de modo que una anotación del *framework* en el dominio no compile. En Go y Python la regla se verifica por revisión de importaciones.

5.3.4. La entidad de dominio **nunca se serializa directamente**: el adaptador HTTP la traduce a un objeto de respuesta. Renombrar un campo del dominio no debe romper a ningún cliente.

5.3.5. Contrato común. Todo servicio respeta las convenciones de `07-api/contracts/openapi/_shared.yaml` del marco de gobernanza:

Aspecto	Regla
Nombres	JSON en camelCase
Identificadores	UUID
Dinero	Entero en unidades menores (<code>totalCents</code>); nunca punto flotante
Fechas	RFC 3339, en UTC
Errores	Sobre único <code>{"error": CÓDIGO, "message", "details"?, "traceId"}</code> , también para rutas inexistentes y JSON malformado
Códigos	VALIDATION_ERROR 400 · UNAUTHORIZED 401 · FORBIDDEN 403 · NOT_FOUND 404 · INVALID_STATUS_TRANSITION y BUSINESS_RULE_VIOLATION 422 · INTERNAL_ERROR 500

Un error nunca expone el mensaje de un *driver*, una traza de pila ni un nombre de servidor interno. El detalle completo va al registro, con su `traceId`.

5.3.6. Listados acotados. Todo listado es paginado: `page` (desde 1) y `limit` (1 a 100, por defecto 20), respuesta `{"data": [...], "meta": {page, limit, total, totalPages}}` y orden estable (los más recientes primero). Un listado sin límite es un defecto, no una simplificación.



5.3.7. Validación del token en el servicio. Cada servicio valida el JWT por sí mismo (numeral 5.6.2): algoritmo RS256 con la **clave pública** del servicio de identidad, rechazo explícito de cualquier otro algoritmo —incluidos none y HS256—, y exp y sub obligatorios. Ningún servicio posee la clave privada ni una clave compartida.

5.3.8. Creación idempotente. Toda operación que crea exige la cabecera Idempotency-Key (8 a 128 caracteres). Repetir la petición con la misma clave devuelve el recurso ya creado (200) sin crear otro. La clave y el recurso se escriben en la **misma transacción**.

5.3.9. Correlación. El servicio reutiliza el X-Correlation-Id recibido o genera uno, lo devuelve en la respuesta, lo escribe en cada línea de su registro estructurado (JSON) y lo usa como traceId del sobre de error.

5.3.10. Límites explícitos. Se declaran en la raíz de composición, con su valor: tiempos máximos del servidor (lectura de cabeceras, lectura, escritura, inactividad), tamaño del *pool* de conexiones, espera máxima por una conexión, tiempo máximo por sentencia y apagado ordenado. Un valor por defecto no declarado no es una decisión: en varios lenguajes significa "sin límite".

5.3.11. Eventos. Un servicio que publica un evento como consecuencia de un cambio en su base lo escribe en una tabla de bandeja de salida (*outbox*) **en la misma transacción** del cambio; un proceso aparte lo publica después. Publicar justo después de confirmar la transacción pierde el evento cuando el proceso cae entre las dos acciones.

5.3.12. Verificación de contrato. El docente verifica cada *-api* por HTTP, sin conocer su lenguaje, con las comprobaciones que lista el Anexo C: autenticación, sobre de error, correlación, validación, idempotencia, transiciones y paginación. Cada equipo automatiza esas mismas comprobaciones en sus pruebas HTTP, adaptadas a los recursos de su dominio, y las mantiene en verde: es la prueba de que piezas escritas en lenguajes distintos se comportan igual.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



5.4. Repositorio -portal y -app

Artefacto	Exigencia
Pantallas del dominio	Solo las de su dominio
Publicación como remoto	Consumible por el contenedor
Consumo del contrato	A través del cliente compartido del contenedor
Estados de cada vista	Cargando, error con reintento, vacío y con datos: los cuatro diseñados
Formularios	Etiqueta asociada a cada campo, error junto al campo, botón deshabilitado mientras hay un envío pendiente

5.4.1. No se implementa aquí el cliente HTTP ni el manejo de sesión o token. Ambos se consumen del repositorio -front. Duplicarlos constituye falta grave.

5.4.2. Toda creación envía Idempotency-Key, y la **reutiliza** mientras se reintenta el mismo dato: un doble clic o un reintento tras un *timeout* no crean dos registros.

5.4.3. El dinero se convierte desde el texto que escribió la persona, nunca multiplicando un número de punto flotante ($0.07 * 100$ no es 7).

5.5. Repositorio -front

Artefacto	Exigencia
Cliente HTTP compartido	Único punto de salida hacia el gateway
Manejo de sesión y token	Centralizado
Composición	Registro y montaje de los remotos de dominio
Navegación y disposición	Común a todo el sistema, con ruta de "página no encontrada"
Errores	Un solo lugar decide el mensaje que ve la persona según el estado HTTP, con la referencia <code>traceId</code>
Límites por petición	Tiempo máximo por petición y X-Correlation-Id en cada una





Aislamiento de remotos

Un portal caído muestra su error en su área; el resto de la aplicación sigue funcionando

5.5.1. Versiones con soporte vigente durante todo el periodo: React 19 o Angular 21, sobre Node 22 LTS o 24. Una versión sin soporte no recibe correcciones de seguridad.

5.5.2. Mientras no exista el dominio de identidad, el contenedor ofrece un inicio de sesión **de desarrollo** que acepta un token emitido con `<abbr>-infra/scripts/dev-token.sh`. Se reemplaza por el portal de identidad; nunca llega a main.

5.6. Repositorio -api-gateway

Estructura y reglas: **Anexo F**.

Artefacto	Exigencia
Configuración de enrutamiento	Un archivo de rutas por dominio expuesto
Filtro de credencial	Rechaza una ruta protegida que llega sin credencial
Límites de tasa y CORS	Configurados explícitamente
Resolución por petición	Un dominio caído no debe tumbar el gateway
Correlación	Reutiliza o genera X-Correlation-Id, lo reenvía al servicio, lo devuelve y lo registra
Errores propios	Con el sobre común: 401 UNAUTHORIZED, 404 NOT_FOUND, 429 TOO_MANY_REQUESTS (con Retry-After), 503 SERVICE_UNAVAILABLE
Pruebas de humo	Salud, ruta desconocida, ruta protegida sin credencial, correlación y CORS

5.6.1. Ningún cliente accede directamente a un servicio de dominio. Toda petición externa atraviesa el gateway. Por eso **solo el gateway se publica al host**; los servicios son alcanzables únicamente en la red interna.





5.6.2. El gateway **filtra** la presencia de credencial; **no la valida**. La validación del token —firma, expiración, permisos— ocurre **en cada servicio**, porque las llamadas internas entre servicios no pasan por el gateway. Un servicio que confía en que "el gateway ya validó" queda sin protección ante cualquier llamada interna.

5.6.3. El gateway resuelve los servicios **en cada petición**, no una sola vez al arrancar. Si un dominio cae, su ruta responde 503 SERVICE_UNAVAILABLE con el sobre común y todas las demás siguen funcionando.

5.6.4. CORS admite únicamente los orígenes del contenedor; permite las cabeceras Authorization, Content-Type, Idempotency-Key y X-Correlation-Id, y expone X-Correlation-Id y Location. Nunca * junto con credenciales.

5.7. Repositorio -worker

Artefacto	Exigencia
Consumidores	Uno por tipo de evento o mensaje atendido
Política de reintento	Explícita, con límite y destino de fallos
Idempotencia	Documentada y verificable
Credencial propia	Token de servicio emitido por el servicio de identidad, nunca versionado
Corridas acotadas	Tamaño máximo de lote y tiempo máximo por corrida; lo pendiente queda para la siguiente

5.7.1. No expone interfaz HTTP de negocio. Solo se admite un punto de estado de salud.

5.7.2. Todo reintento es **acotado**, con espera exponencial y **variación aleatoria (jitter)**, y solo ante errores que un reintento puede corregir: fallas de red, 429 y 5xx. Un 4xx no se reintenta. Sin variación, muchos procesos que fallan juntos reintentan juntos y prolongan la caída.



5.7.3. Cada corrida tiene su propio X-Correlation-Id, enviado en cada llamada: una corrida se sigue de punta a punta en los registros de todo el sistema.

5.8. Repositorio -workflow

Artefacto	Exigencia
Definición del proceso	Pasos, orden y condiciones
Compensaciones	Qué se deshace ante el fallo de cada paso
Registro de ejecución	Trazable por identificador de correlación
Contrato de entrada	El mismo de un -api: valida el token, valida la entrada, responde con el sobre común
Inicio idempotente	Idempotency-Key obligatoria: repetir la petición devuelve la misma saga y no ejecuta nada de nuevo
Pasos idempotentes	Cada paso envía Idempotency-Key: <saga>:<paso>; cada compensación es idempotente
Estado	Persistido después de cada paso

5.8.1. Un proceso contenido dentro de un solo dominio no pertenece a este repositorio, sino al -api de ese dominio.

5.8.2. Las llamadas a los participantes usan la credencial de servicio del *workflow*, no el token de la persona: ese token puede vencer en medio de una compensación.

5.8.3. La respuesta indica el paso que falló (*failedStep*); el detalle del fallo —servidores, códigos— queda en el estado de la saga y en el registro, nunca en la respuesta.

5.8.4. Dónde se persiste el estado de las sagas en producción es una decisión del equipo, registrada en un ADR (numeral 4.2.3). La topología no asigna un -db a los transversales.



5.9. Repositorio -infra

Artefacto	Exigencia
Composición raíz	Levanta el sistema completo
Red y variables	Definidas de forma explícita
Secretos	Referenciados, nunca versionados
Observabilidad	Registro estructurado, métricas y trazas
Claves de desarrollo	scripts/dev-keys.sh y scripts/dev-token.sh, solo para develop; keys/ y .env ignorados por git
Arranque documentado	Levantar, migrar cada -db, verificar por el gateway

5.9.1. Las bases de datos no se definen aquí. Este repositorio las compone; cada -db las contiene.

5.9.2. En qa y main, la clave pública y los tokens de servicio provienen del servicio de identidad, como secretos del ambiente. Las claves de desarrollo nunca salen de develop.

6. Régimen de ramas

6.1. Repositorios de categoría B

6.1.1. Una sola rama permanente: main.

6.1.2. Todo cambio entra por una rama hija con prefijo docs/, mediante Pull Request.

6.1.3. El Pull Request requiere una aprobación del docente.



6.2. Repositorios de categoría C

6.2.1. Tres ramas permanentes, correspondientes a tres ambientes:

```
develop  <--- PR --- feat/...  fix/...  chore/...
qa       <--- PR --- qa/...
main     <--- PR --- release/... hotfix/...
```

6.2.2. Ninguna rama permanente acepta un commit directo. La restricción está aplicada a nivel de organización: el intento falla con error, no es una recomendación.

6.2.3. Cada rama padre se alimenta exclusivamente de ramas hijas suyas. Para ingresar a qa se corta una rama de qa; para ingresar a main, una rama de main.

6.2.4. No existe fusión entre ramas permanentes. Las operaciones merge develop → qa y merge qa → main están prohibidas.

6.3. Nomenclatura de ramas hijas

6.3.1. Toda rama hija se nombra <prefijo>/<descripcion-en-kebab-case>.

Rama destino	Prefijos admitidos
develop	feat/, fix/, chore/
qa	qa/
main	release/, hotfix/
main (categoría B)	docs/



6.3.2. Las ramas de release se nombran `release/<mayor>.<menor>.<parche>`, siguiendo versionado semántico.

6.3.3. No se admite ningún otro prefijo. Una rama fuera de esta nomenclatura se considera trabajo no trazable.

6.4. Vida de las ramas hijas

6.4.1. Una rama hija corresponde a **una sola** tarea. No se mezclan dos historias en la misma rama.

6.4.2. La vida de una rama hija no debe exceder **cinco días hábiles**. Una rama más antigua es indicio de trabajo no integrado y se penaliza.

6.4.3. Eliminar la rama tras la fusión es práctica recomendada, no obligatoria.

7. Aislamiento de datos

7.1. Cada dominio tiene **su propia base de datos**, en su propia instancia, con su propio volumen. Un motor compartido con esquemas separados **no** cumple esta regla.

7.2. Cada base es dueña de su estructura. No existe un repositorio central de migraciones.

7.3. **Ningún servicio consulta la base de datos de otro dominio.** Si el dominio A requiere datos del dominio B, los solicita a la interfaz publicada de B.



7.4. La integridad referencial entre dominios no se implementa con llaves foráneas, sino con identificadores y verificación por contrato.

7.5. La consistencia entre dominios es eventual y se resuelve por eventos o por saga, nunca por transacción distribuida sobre varias bases.

8. Commits

8.1. Todo mensaje de commit sigue el estándar Conventional Commits:

```
<tipo><alcance>: <descripcion en minuscula, modo imperativo, sin punto final>  
  
<cuerpo opcional: explica POR QUE, no QUE>  
  
<pie opcional: referencia a la historia de usuario>
```

8.2. Tipos admitidos: feat, fix, docs, style, refactor, test, chore, perf.

8.3. Ejemplos conformes y no conformes:

Mensaje	Estado
feat(catalog): add copy availability check	Conforme
fix(circulation): correct overdue fine calculation	Conforme
Cambios	No conforme: sin tipo, sin alcance, sin descripción



	útil
feat: Arreglo del bug.	No conforme: mayúscula inicial y punto final
avance	No conforme
WIP	No conforme

8.4. Un historial con commits no conformes se penaliza aunque el código sea correcto. El historial es entregable.

9. Pull Requests

9.1. Todo Pull Request declara qué historia de usuario atiende.

9.2. Tamaño máximo: **400 líneas** de cambio, excluyendo pruebas y archivos generados. Un cambio mayor se divide.

9.3. El Pull Request se abre contra la rama que corresponde a su prefijo, según el numeral 6.3.1. Abrirlo contra otra rama es error de proceso.

9.4. En **develop** y **qa** la regla de revisión la define cada equipo. El equipo la acuerda una vez y la registra por escrito en **00-governance/git-conventions.md**. Un equipo sin esa regla escrita se evalúa como sin gobernanza.

9.5. En **main** se exige una aprobación del docente, resolución de todas las conversaciones abiertas y ausencia de aprobaciones obsoletas.



9.6. Está prohibido reescribir el historial de una rama ya publicada (**rebase** sobre rama compartida, **push --force**). La reescritura destruye los rastros de promoción.

9.7. Todo Pull Request de la organización recibe una **revisión automática** escrita en inglés, dentro de la hora siguiente a su apertura:

Tipo de repositorio	Alcance de la revisión
Documentación	completitud, consistencia entre documentos, trazabilidad, nomenclatura e higiene del Pull Request
Código	análisis exigente de brechas: arquitectura desorientada, violaciones de frontera, modos de falla, idempotencia, seguridad, datos, contratos y pruebas, clasificados como Critical, Major o Minor

9.8. La revisión automática **aprueba siempre**. No bloquea el trabajo del equipo: deja sus hallazgos por escrito como **traza**.

9.9. El control humano ocurre en la **sustentación**. Cada hallazgo de la revisión automática se contrasta con lo que el equipo hizo:

Respuesta del equipo	Tratamiento
Lo aplicó, y muestra cómo	se valora
No lo aplicó, con una justificación técnica válida	se acepta
No lo consideró	si el docente confirma el defecto, afecta la dimensión correspondiente de la rúbrica

Una decisión razonada de no seguir una recomendación es aceptable. No haberla considerado, no lo es.



10. Promoción y trazabilidad

Este numeral concentra la diferencia central del modelo y el mayor peso en la evaluación.

10.1. El avance de un cambio entre ambientes se realiza por **re-aplicación**, no por fusión.

10.2. Procedimiento obligatorio para promover de develop a qa:

```
git switch qa && git pull
git switch -c qa/hu-07-renovacion-prestamo
git cherry-pick -x <sha-del-commit-en-develop>
git push -u origin qa/hu-07-renovacion-prestamo
```

10.3. La opción -x es **obligatoria**. Al re-aplicar, el commit recibe un identificador nuevo; la línea (cherry picked from commit <sha>) es el único vínculo verificable entre las dos versiones del mismo cambio.

10.4. Un commit presente en qa o en main que no exhiba el rastro de promoción se considera **no trazable**, y el trabajo que contiene no se contabiliza como avance, aunque funcione.

10.5. El identificador citado en el rastro debe existir efectivamente en la rama de origen. Un rastro que apunte a un identificador inexistente se considera falsificación de evidencia y se trata según el numeral 13.



11. Release

11.1. La rama de release se corta de main y nace vacía.

11.2. Se llena de forma incremental: **un commit por cada historia de usuario ya validada** en qa, sea por re-aplicación o por commit nuevo controlado.

```
main ---+----- PR ---> main
|
+-- release/1.2.0
    |-- commit <-- HU-07 (validada en qa)
    |-- commit <-- HU-09 (validada en qa)
    +-- commit <-- HU-11 (validada en qa)
```

11.3. El Pull Request de la release contiene obligatoriamente:

Elemento	Exigencia
Listado de historias	Identificador y título de cada una
Rastro por historia	El cherry picked from correspondiente
Alcance declarado	Qué incluye y qué se dejó fuera
Instrucciones de despliegue	Migraciones a ejecutar, variables nuevas
Plan de reversión	Cómo se vuelve atrás si falla

11.4. Cadencia por defecto: una release por corte, en las semanas 5, 10 y 15. Una cadencia distinta debe estar acordada y registrada en 00-governance antes del corte.

11.5. Fusionada la release, se etiqueta con versionado semántico:

```
git switch main && git pull
git tag -a v1.2.0 -m "Release v1.2.0: alcance resumido"
git push origin v1.2.0
```



11.6. La única otra rama hija de `main` admitida es `hotfix/`, para corrección urgente en producción. Todo `hotfix` se re-aplica después a `qa` y a `develop`, para que las ramas no diverjan.

12. Reglas de oro

Las diecisiete reglas anteriores se resumen en diez enunciados. Si alguno se incumple, hay una regla del articulado que lo respalda y una verificación del numeral 15 que lo demuestra.

- 1. Ninguna rama permanente acepta un commit directo.** Ni `main`, ni `qa`, ni `develop`. Está bloqueado por la plataforma: el intento falla.
- 2. Cada rama padre se alimenta solo de hijas suyas.** Para entrar a `qa`, una rama de `qa`. Para entrar a `main`, una `release`.
- 3. Nunca se fusiona una rama permanente en otra.** La promoción es re-aplicación, no fusión.
- 4. Toda promoción deja rastro:** `git cherry-pick -x`. Sin el rastro no hay trazabilidad, y sin trazabilidad el trabajo no se contabiliza.
- 5. Una rama, una tarea, cinco días.** Ramas cortas que se integran rápido.
- 6. La rama principal es producción:** `main`. Solo se toca con una `release` o un `hotfix`, siempre con aprobación del docente.



7. La estructura de la base vive en su repositorio de datos, el -db. Nunca en el -api.
8. Cada dominio consulta únicamente su propia base. Si A necesita datos de B, se los pide a la interfaz de B. Jamás a su base.
9. El cliente HTTP y la sesión viven en el contenedor, no repetidos en cada portal.
10. Lo que no está publicado, no existe. El avance se mide sobre el repositorio remoto, no sobre la máquina de nadie.

13. Faltas graves

Las siguientes conductas se sancionan con independencia del avance funcional alcanzado.

Falta	Efecto
Fusionar una rama permanente en otra	Anula la dimensión de trazabilidad del corte
Re-aplicar sin -x	El trabajo promovido no se contabiliza
Rastro de promoción que apunta a un identificador inexistente	Falsificación de evidencia. Se reporta
Consultar la base de datos de otro dominio	Anula la dimensión de aislamiento y arquitectura
Versionar secretos o credenciales reales	Se exige rotación inmediata y se sanciona
Reescribir el historial de una rama publicada	Anula la trazabilidad del periodo afectado
Alterar o desactivar las reglas de protección	Falta de integridad académica. Se reporta
Nomenclatura fuera del numeral 4.5	Anula la dimensión de topología



14. Qué bloquea la plataforma y qué se audita

La distinción importa: lo que la plataforma impide no puede ocurrir; lo que se audita depende de la disciplina del equipo y es donde se pierde o se gana la nota.

14.1. Bloqueado automáticamente

Regla	Mecanismo
Commit directo a main, qa o develop	Regla de organización
Fusión sin Pull Request	Regla de organización
Aprobación distinta a la del docente en main	CODEOWNERS + revisión obligatoria de propietario
Eliminación de una rama permanente	Regla de organización
Reescritura forzada del historial	Regla de organización
Fusión con conversaciones sin resolver	Regla de organización

14.2. Auditado por el docente

Regla	Cómo se verifica
Promoción por re-aplicación	Inspección del historial
Presencia del rastro -x	Búsqueda en los mensajes de commit
Correspondencia del rastro con el origen	Verificación del identificador citado
Nomenclatura de repositorios y ramas	Listado y contraste contra la norma
Formato de los mensajes de commit	Búsqueda por expresión regular
Vida de las ramas hijas	Fecha del último commit por rama
Aislamiento de datos	Revisión de configuración y de cadenas de conexión
Contenido mínimo por repositorio	Listado de archivos
Tamaño de los Pull Requests	Estadísticas del Pull Request
Correspondencia tablero contra historial	Contraste entre historias cerradas y commits



15. Comandos de auditoría

Estos son los comandos que se ejecutan para evaluar. Se publican deliberadamente: el equipo puede y debe correrlos sobre sí mismo antes de cada corte.

15.1. Ramas fuera de la nomenclatura

```
git branch -r --format='%(%(refname:short)' \
| grep -vE '^origin/(develop|qa|main)$' \
| grep -vE '^origin/(feat|fix|chore|qa|release|hotfix|docs)/'
```

Cualquier línea devuelta es un incumplimiento del numeral 6.3.

15.2. Mensajes de commit no conformes

```
git log --format='%s' origin/develop \
| grep -vE '^(feat|fix|docs|style|refactor|test|chore|perf)(\[a-z0-9.-]+\))?: [a-z]'
```

Cualquier línea devuelta es un incumplimiento del numeral 8.

15.3. Commits en qa sin rastro de promoción

```
git log origin/qa --not origin/main --format='%H %s' > /tmp/en-qa.txt
git log origin/qa --not origin/main --grep='cherry picked from' --format='%H' > /tmp/con-
rastros.txt
# Los commits de en-qa.txt que no aparezcan en con-rastros.txt
# son trabajo no trazable (numeral 10.4)
```



15.4. Fusiones entre ramas permanentes

```
git log origin/qa --merges --format='%h %s' | grep -iE 'develop|qa|main'
```

Cualquier resultado que corresponda a la fusión de una rama permanente en otra es falta grave (numeral 13).

15.5. Ramas hijas estancadas

```
git for-each-ref --sort=committerdate refs/remotes/origin \  
--format='%(committerdate:short) %(refname:short)'
```

Ramas hijas cuyo último commit supere cinco días hábiles incumplen el numeral 6.4.2.

15.6. Verificación de un rastro de promoción

```
# Extraer el identificador citado y comprobar que existe en origen  
git log origin/main --format='%H%n%b' | grep -o 'cherry picked from commit [0-9a-f]*'  
git cat-file -e <sha-citado> && echo "existe" || echo "NO EXISTE - numeral 13"
```

15.7. Aislamiento de datos

```
grep -rniE 'host=|DB_HOST|DATABASE_URL|jdbc:|postgres://' . \  
--include='*.env*' --include='*.yaml' --include='*.yml' --include='*.json'
```

Un servicio cuya configuración apunte a la base de otro dominio incumple el numeral 7.3.

15.8. Secretos versionados



```
git log -p --all | grep -inE 'password\s*=\|secret\s*=\|api[_-]?key\s*=\|BEGIN (RSA|PRIVATE) KEY'
```

Cualquier resultado con un valor real es falta grave (numeral 13).

16. Rúbrica de evaluación

Cada corte se califica sobre 100 puntos distribuidos en siete dimensiones. La calificación se obtiene ejecutando las verificaciones del numeral 15 sobre los repositorios publicados.

#	Dimensión	Puntos
1	Topología y nomenclatura	15
2	Régimen de ramas	15
3	Trazabilidad de la promoción	20
4	Commits y Pull Requests	10
5	Aislamiento de datos y arquitectura	20
6	Release y evidencia de entrega	15
7	Tablero y gobernanza documental	5

16.1. Dimensión 1 — Topología y nomenclatura (15)

Criterio	Puntos
Existen todos los repositorios obligatorios de la fórmula (4.1); los adicionales están justificados en un ADR (4.2.1)	4
Los cinco transversales existen y cada uno cumple su contenido mínimo	4
Cada dominio tiene sus piezas completas	4





Cero incumplimientos de nomenclatura (numeral 4.5)	3
--	---

16.2. Dimensión 2 — Régimen de ramas (15)

Criterio	Puntos
Las tres ramas permanentes existen en todos los repositorios	3
Cero ramas fuera de la nomenclatura (verificación 15.1)	4
Cada rama padre alimentada solo por hijas suyas	4
Cero ramas hijas estancadas (verificación 15.5)	4

16.3. Dimensión 3 — Trazabilidad de la promoción (20)

Criterio	Puntos
Cero fusiones entre ramas permanentes (verificación 15.4)	6
Todo commit promovido exhibe rastro -x (verificación 15.3)	8
Todo rastro apunta a un identificador existente (verificación 15.6)	6

16.4. Dimensión 4 — Commits y Pull Requests (10)

Criterio	Puntos
Mensajes conformes al numeral 8 (verificación 15.2)	4
Pull Requests dentro del límite de tamaño	3
Cada Pull Request referencia su historia de usuario	3

16.5. Dimensión 5 — Aislamiento de datos y arquitectura (20)



Criterio	Puntos
Una base por dominio, en instancias separadas	4
Cero accesos cruzados entre bases (verificación 15.7)	4
El esquema vive en el -db, no en el -api	3
El dominio no depende de framework ni de persistencia	3
Cliente HTTP y sesión centralizados en el contenedor	2
Los servicios cumplen el contrato común (5.3.5 a 5.3.9) y las comprobaciones del Anexo C	4

16.6. Dimensión 6 — Release y evidencia de entrega (15)

Criterio	Puntos
La release se construyó de forma incremental	4
El Pull Request contiene los cinco elementos del numeral 11.3	5
Etiqueta de versión publicada	2
El sistema levanta desde cero siguiendo el -infra	4

16.7. Dimensión 7 — Tablero y gobernanza documental (5)

Criterio	Puntos
El estado del tablero corresponde al historial real	3
La regla de revisión del equipo está escrita en 00-governance	2

16.8. Penalizaciones

16.8.1. Cada falta grave del numeral 13 anula la totalidad de los puntos de la dimensión afectada, sin perjuicio de las medidas adicionales que correspondan.



16.8.2. Un repositorio de categoría C que no exista, o que exista vacío, resta la totalidad de los puntos proporcionales que le corresponden en la dimensión 1.

16.8.3. El trabajo no publicado en el repositorio remoto no se evalúa (numeral 1.4).

17. Autoevaluación previa al corte

Ejecute esta lista sobre sus repositorios **antes** de la fecha de entrega. Cada punto corresponde a una verificación real de la rúbrica.

- ☐ Existen todos los repositorios que exige la fórmula del numeral 4.1
- ☐ Todo repositorio adicional tiene su ADR, y las decisiones de lenguaje, motor y *framework* también
- ☐ Ningún nombre incurre en las formas prohibidas del numeral 4.5.4
- ☐ Todos los repositorios tienen README.md, CODEOWNERS, .gitignore, .env.example y ci.yml
- ☐ El flujo de CI está en verde en develop de cada repositorio de código
- ☐ Cada ADR tiene criterio dominante y costo aceptado (numeral 4.2.3)
- ☐ Cada -api cumple las comprobaciones de contrato del Anexo C (numeral 5.3.12)
- ☐ Todo listado es paginado y toda creación exige Idempotency-Key
- ☐ Ningún repositorio versiona claves, tokens ni .env
- ☐ Todos tienen sus tres ramas permanentes
- ☐ La verificación 15.1 no devuelve ninguna línea
- ☐ La verificación 15.2 no devuelve ninguna línea

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL – SNIES 2828

- ☐ La verificación 15.3 no arroja commits sin rastro
- ☐ La verificación 15.4 no arroja fusiones entre permanentes
- ☐ La verificación 15.5 no arroja ramas estancadas
- ☐ La verificación 15.7 no arroja accesos cruzados
- ☐ La verificación 15.8 no arroja secretos
- ☐ Cada dominio tiene su base propia, con volumen propio
- ☐ Ningún -api conserva migraciones
- ☐ Ningún -portal o -app implementa su propio manejo de token
- ☐ El Pull Request de la release contiene los cinco elementos exigidos
- ☐ La regla de revisión del equipo está escrita en 00-governance
- ☐ El tablero refleja el estado real del trabajo
- ☐ Cada hallazgo de la revisión automática tiene respuesta: aplicado, o no aplicado con justificación (numeral 9.9)

18. Vigencia

18.1. Esta norma rige desde su publicación y hasta el cierre del periodo académico 2026-B.

18.2. Las reglas del numeral 14.1 se encuentran aplicadas y operativas en la organización.

18.3. Cada repositorio de código incluye su flujo de integración continua (ci.yml, Anexo I), y cada -api automatiza las comprobaciones de contrato del Anexo C (numeral 5.3.12). Su incorporación como verificación **bloqueante** en las reglas de la organización se comunicará conforme al numeral

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



18.4; hasta entonces, su resultado se audita conforme al numeral 14.2, y un flujo en rojo en la rama de destino de una release es evidencia en contra en la dimensión 6.

18.4. Cualquier ajuste a esta norma se comunicará por escrito y con anterioridad al corte sobre el cual tenga efecto.

19. Anexos

Cada anexo describe un tipo de repositorio: su propósito, la **estructura esperada**, qué va en cada parte, sus reglas, las decisiones que el equipo registra en un ADR y **cómo se verifica**. Los árboles de directorios de los anexos no son ilustrativos: provienen de implementaciones completas que se construyeron y verificaron de punta a punta antes de publicar esta norma.

Anexo	Repositorio	Tecnologías
A	-db relacional	PostgreSQL + Liquibase o Flyway
B	-db documental	MongoDB + Liquibase
C	-api hexagonal	Go · Java · Python · C#
D	-worker	Go · Java · Python · C#
E	-workflow (sagas)	Go · Java · Python · C#
F	-api-gateway	NGINX
G	-infra	Docker Compose + observabilidad
H	-front y -portal	React · Angular
I	Archivos comunes	CODEOWNERS, formato de Pull Request, integración continua, seguimiento de ambiente

19.1. Todos los anexos usan un mismo dominio de ejemplo —pedidos (orders)— con **el mismo contrato público** en todos los lenguajes. Al construir su proyecto, el equipo reemplaza el ejemplo



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL – SNIES 2828

por sus dominios y conserva la forma: las capas, la regla de dependencia, el contrato común y los límites explícitos.

Jesús Ariel González Bonilla

Docente Ingeniería de Sistemas — Corporación Universitaria del Huila, CORHUILA

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 – 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 – 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co
Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"